3.2. The Main Model

(1) **Regression Tree** absorbs all the observations and starts with the entire set. The prediction of each node is the average value of the node. It is the major difference between Regression Tree and Classification Tree.

And the division of a node is still by a binary split of the most "efficient" feature, decided by square error, which is of the similar usage as entropy in classification. We will say the split feature as the most efficient if it provides the least square error.

(2) **Gradient Tree Boosting Algorithm**, as shown in Figure 7.

Algorithm 10.3 *Gradient Tree Boosting Algorithm.*

1. Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.
2. For $m = 1$ to M:
   (a) For $i = 1, 2, \ldots, N$ compute
   $$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}.$$
   (b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm}, $j = 1, 2, \ldots, J_m$.
   (c) For $j = 1, 2, \ldots, J_m$ compute
   $$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L\left(y_i, f_{m-1}(x_i) + \gamma\right).$$
   (d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.
3. Output $\hat{f}(x) = f_M(x)$.

Figure 7. Gradient Tree Boosting Algorithm

Note that

- N: number of observations
- M: number of iterations
- R_{jm}: terminal regions in mth iteration
- $f_m(x)$: the regression tree in mth iteration
- L: loss function, e.g. square error loss
- r_{im}: residual in the mth iteration and ith observation
- γ_{jm}: shrinkage parameter
- $\hat{f}(x)$: the final prediction model

(3) **Random Forest**, as shown in Figure 8.

Note that B means the turns of bootstrap.

Algorithm 15.1 *Random Forest for Regression or Classification.*

1. For $b = 1$ to B:
   (a) Draw a bootstrap sample $\mathbf{Z}^*$ of size N from the training data.
   (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
      i. Select m variables at random from the p variables.
      ii. Pick the best variable/split-point among the m.
      iii. Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$.

To make a prediction at a new point x:

Regression: $\hat{f}_{\text{rf}}^B(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$.

Classification: Let $\hat{C}_b(x)$ be the class prediction of the bth random-forest tree. Then $\hat{C}_{\text{rf}}^B(x) = \textit{majority vote } \{\hat{C}_b(x)\}_1^B$.

Figure 8. Gradient Tree Boosting Algorithm

3.3. Hyperparameters of GBDT

In this algorithm (especially in this R function of the package "gbm"), we have 3 hyperparameters to tune.

Shrinkage: a factor between 0 and 1. It will shrink the significance of the next tree in order to avoid overfitting.

n.trees and interaction.depth: parameters related to pruning and at the same time, we have cross-validation to validate the model performance.

3.4. Compare GBDT with Random Forest

Random Forest uses bagging with even bootstrapping, which is basically decreasing the variance over features. By contrast, GBDT uses boosting to focus on wrong estimates to control the possibility of biasedness. There are two essential differences between bagging and boosting.

- Bagging uses bootstrap method and calculate the average value of all predictions. On one hand, average will not change the expectations of the estimate because the distribution of all bootstrap subsets is the empirical distribution of the whole dataset. So the identical distribution ensures $\boldsymbol{E}[\frac{\sum \boldsymbol{X_i}}{\boldsymbol{n}}] = \boldsymbol{E}[\boldsymbol{X_i}]$. On the other hand, the variance of all the subsets will decrease because of $\boldsymbol{Var}[\frac{\sum \boldsymbol{X_i}}{\boldsymbol{n}}] = \boldsymbol{Var}[\boldsymbol{X_i}]$. It holds based on the fact that the bootstrap subsets are basically independent so $\boldsymbol{Var}[\frac{\sum \boldsymbol{X_i}}{\boldsymbol{n}}] = \boldsymbol{Var}[\boldsymbol{X_i}]$ can expand using independence property of variance. So bagging scheme can optimize better variance-wise.